Grado en Ingeniería Informática

Tecnologías de Desarrollo de Software

# Arquitectura de la aplicación

Facultad de Informática

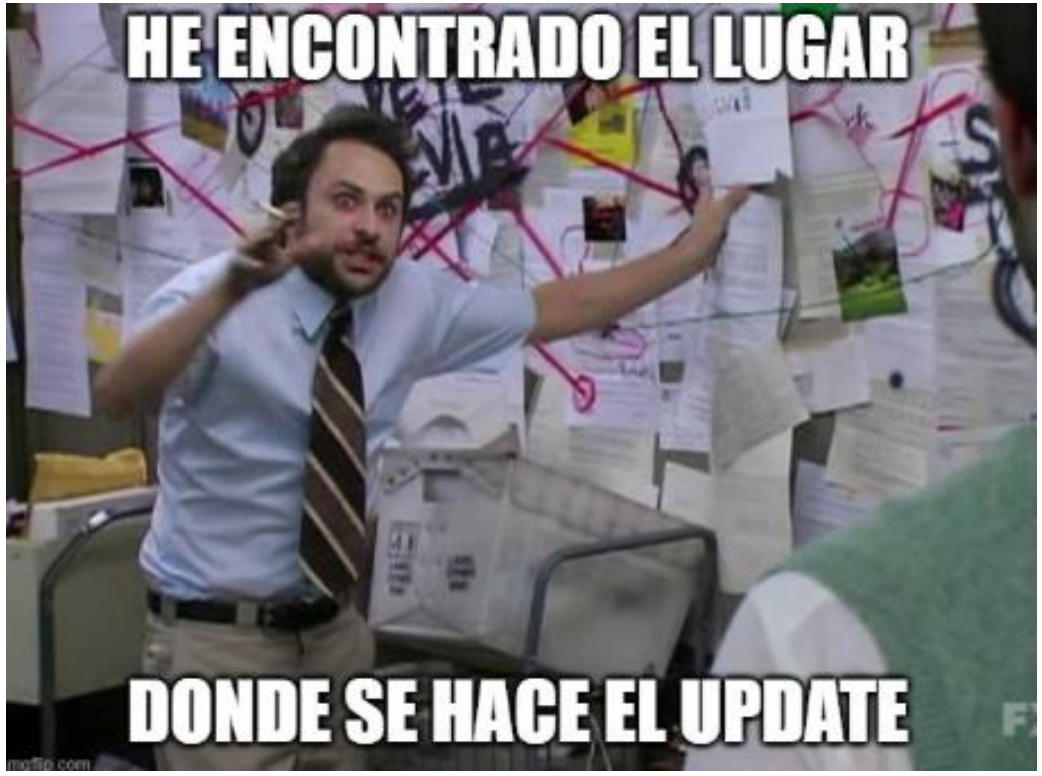
# Motivación

Ya sé POO y patrones. ¿Para qué quiero aprender arquitecturas de aplicación?

# Motivación

Ya sé POO y patrones. ¿Para qué quiero aprender arquitecturas de aplicación?

- Objetivo típico de una arquitectura: reducir al máximo las dependencias entre distintos componentes




Fuente: https://imgflip.com/memegenerator



# Motivación

## Ya sé POO y patrones. ¿Para qué quiero aprender arquitecturas de aplicación?

- Objetivo típico de una arquitectura: reducir al máximo las dependencias entre distintos componentes

```
salirMI.setOnAction(e -> {
   // Asumo que al salir quiero guardar el producto que he vendido
   double precio = Double.parseDouble(precioTF.getText());
   int cantidad = Integer.parseInt(cantidadTF.getText());

   // Calculo total menos descuento típico
   double total = precio * cantidad - precio * cantidad * 0.15;

   // Me conecto a la base de datos
   Connection conn = DriverManager.getConnection("jdbc:mysql://…");

   // Actualizo la venta
   Statement stmt = conn.createStatement();
   stmt.executeUpdate("UPDATE Venta SET total = "+total+" WHERE id = "+idVenta);
   stmt.close();
   conn.close();
   // Ya puedo salir tranquilo
   Platform.exit();
});
```

Dejando de lado otros problemas (seguridad, mal modelado de la bbdd/dominio, etc.):

- ¿Y si quiero hacer el “UPDATE” desde otro evento?
- ¿Y si cambia el % de descuento?
- ¿Y si cambia la base de datos?
- ¿Y si pasamos a interfaz web?
- ¿Y si queremos hacer tests unitarios?

# Motivación

## Ya sé POO y patrones. ¿Para qué quiero aprender arquitecturas de aplicación?

- Objetivo típico de una arquitectura: reducir al máximo las dependencias entre distintos componentes

```
salirMI.setOnAction(e -> {
   // Asumo que al salir quiero guardar el producto que he vendido
   double precio = Double.parseDouble(precioTF.getText());
   int cantidad = Integer.parseInt(cantidadTF.getText());

   // Calculo total menos descuento típico
   double total = precio * cantidad - precio * cantidad * 0.15;

   // Me conecto a la base de datos
   Connection conn = DriverManager.getConnection("jdbc:mysql://…");

   // Actualizo la venta
   Statement stmt = conn.createStatement();
   stmt.executeUpdate("UPDATE Venta SET total = "+total+" WHERE id = "+idVenta);
   stmt.close();
   conn.close();
   // Ya puedo salir tranquilo
   Platform.exit();
});
```



Fuente: https://imgflip.com/memegenerator

# Motivación

Ya sé POO y patrones. ¿Para qué quiero aprender arquitecturas de aplicación?

- Objetivo típico de una arquitectura: reducir al máximo las dependencias entre distintos componentes

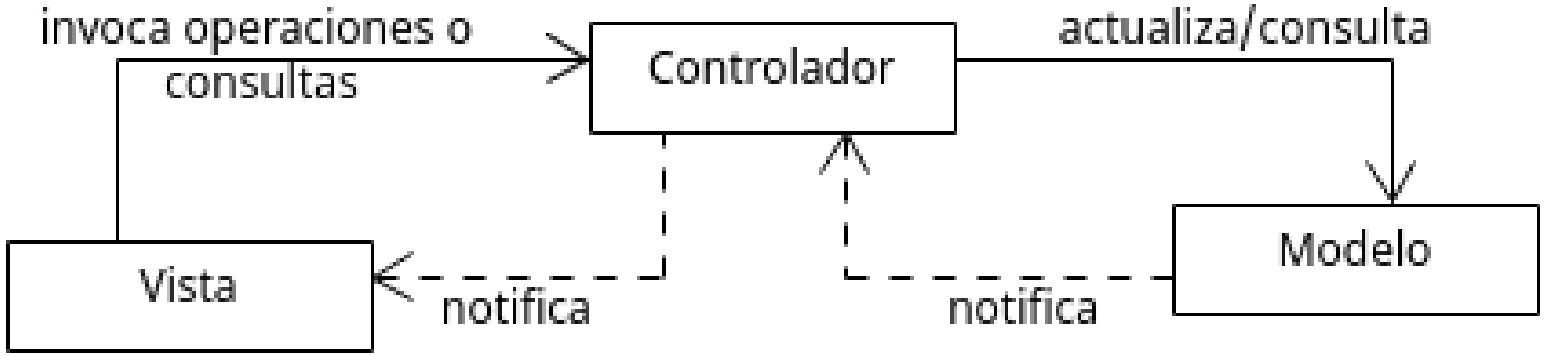
- Ventajas
  - **Mantenimiento/Escalabilidad**: Podemos cambiar fácilmente un componente (interfaz, bbdd, etc.) sin reescribirlo todo.
  - **Reutilización**: Podemos utilizar partes del mismo programa en variaciones (varias interfaces, varios tipos de persistencia, etc.).
  - **Testeabilidad**: Podemos probar por separado los distintos componentes, consiguiendo tests más unitarios y precisos.
  - Etc.

- Son un paso más en el desarrollo de software de calidad: Las arquitecturas se basan en patrones de diseño, que se basan en propiedades de OO.

# Arquitectura MVC

## Arquitectura Modelo-Vista-Controlador

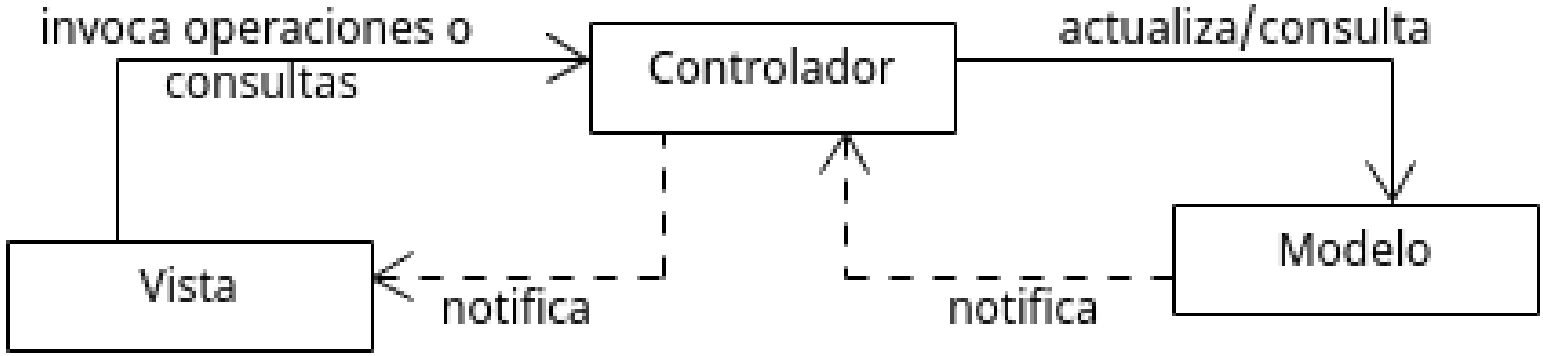
- De los primeros modelos de arquitectura software existentes.
- Muchas variantes (p. ej. uso de Data Transfer Objects (DTOs)).
- Inspiración para otras arquitecturas:
  - Separación de responsabilidades.
  - Independencia de lógica de negocio/interfaz.



# Arquitectura MVC

## Ejemplo de aplicación – TPV en JavaFX

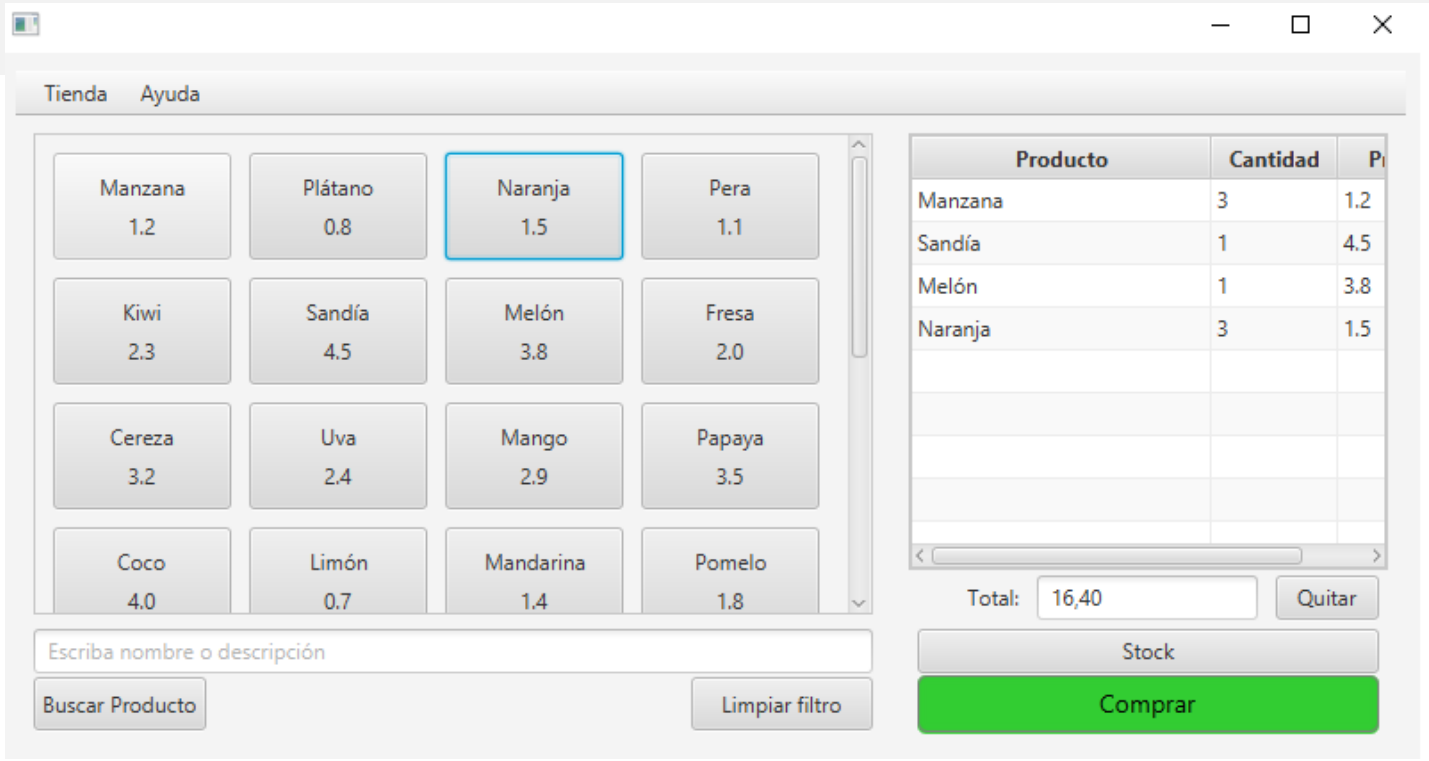
- Aplicaremos MVC usando lo visto hasta ahora de JavaFX.
- Seguiremos los pasos/ejercicios del boletín (en el repo GitHub de la asignatura):



# Ejemplo de aplicación de MVC

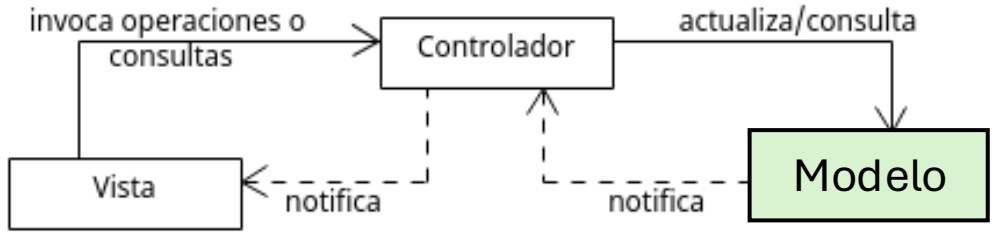
**Ejercicio 1** – Ejecutar y probar la aplicación de TPV

- Obtener el proyecto de GitHub (git clone....)
- Importar el proyecto en Eclipse
- Ejecutar el proyecto
- Jugar un poco con la aplicación

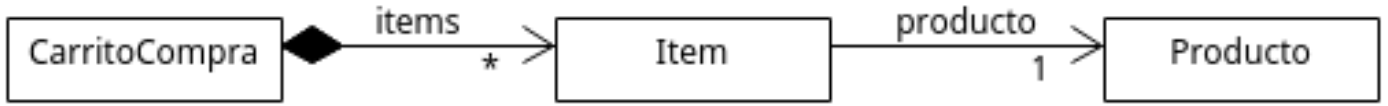


# Arquitectura MVC - Modelo

## **MODELO** - ¿Qué se hace ahí?

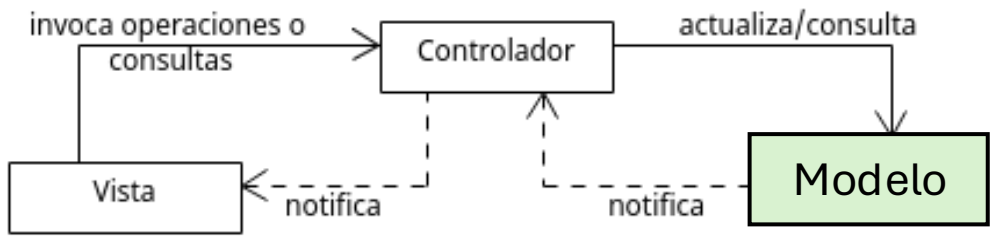



- Representación conceptos del dominio:
  - Datos
    - P.ej. Descripción de un producto, cantidad de una venta, etc.
  - Lógica de negocio
    - P.ej. Cómo calcular el total de una venta
  - Reglas de la aplicación
    - P.ej. No permitir ítems repetidos en un carrito

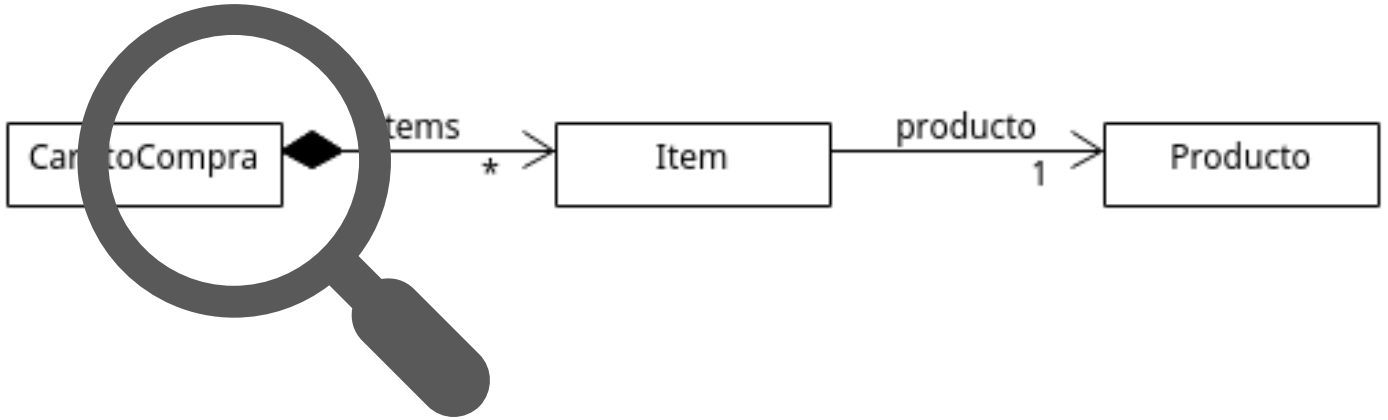


# Arquitectura MVC - Modelo

## **Ejercicio 2** – Vamos a inspeccionar el modelo

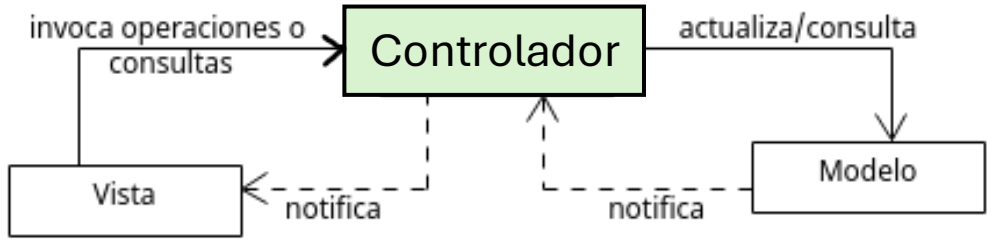



- Se recomienda usar atajos de Eclipse.
- **Objetivo**: Encontrar dónde hay lógica de negocio implementada.



# Arquitectura MVC - Controlador

## CONTROLADOR - ¿Qué se hace ahí?

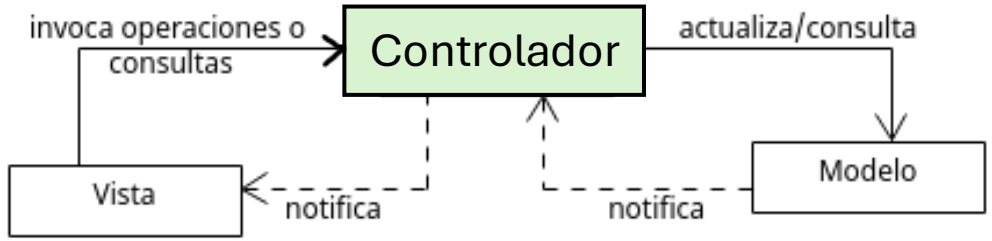



- Intermediario entre vista y modelo.
- Sus métodos son las “operaciones” que puede hacer un usuario.
- Puede comprobar que se cumplen precondiciones.
  - Por ejemplo usando librería Guava – Preconditions, assert, etc.

```
public void aumentarStock(Producto producto, int cantidad) {
   //Aseguramos que el valor de cantidad es siempre positivo
   Preconditions.checkState(cantidad >= 0);
   Log.info("Aumentamos el stock del producto {} en {} unidades",producto.getCodigo(),cantidad);
   this.modificarStock(producto, cantidad);
}
```

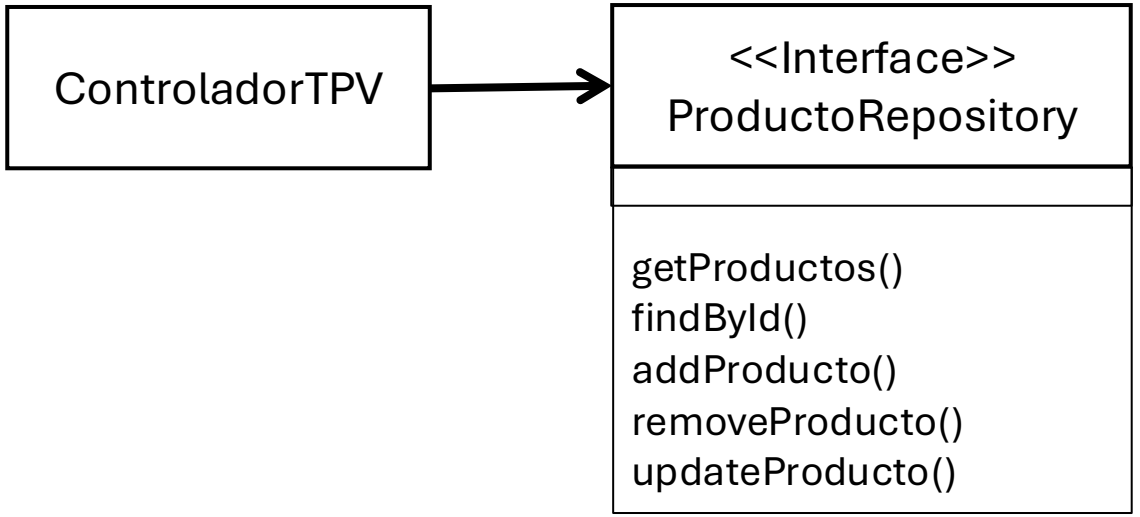
# Arquitectura MVC - Controlador

## **CONTROLADOR** - ¿Qué se hace ahí?



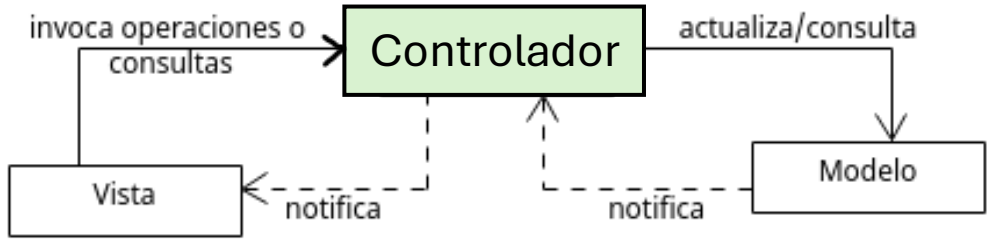

- Se encarga de acceder a la persistencia

**OJO**: no se implementa dentro del controlador sino, por ejemplo, usando el patrón repositorio



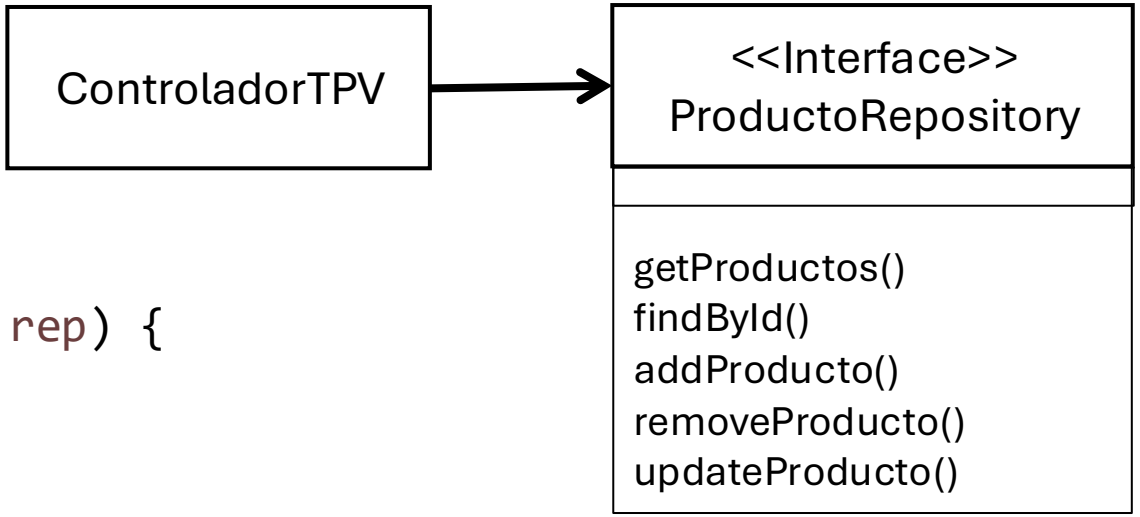
# Arquitectura MVC - Controlador

## **Ejercicio 3** – Diseño del controlador




**Objetivo**: Discutir sobre las siguientes cuestiones:

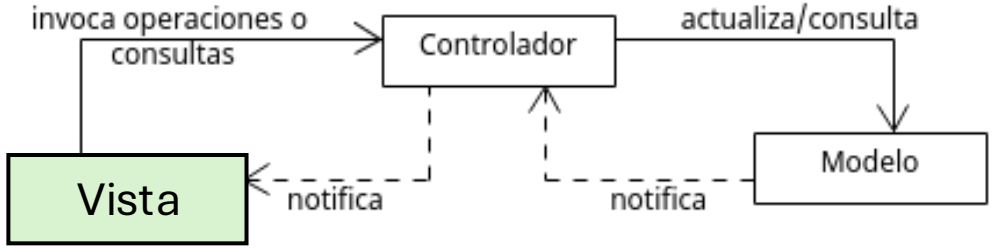
- ¿ Por qué es un interfaz?
- ¿Por qué se pasa como parámetro?




```
public ControladorTPV(ProductoRepository rep) {
    this.repositorioProductos = rep;
    nuevoCarrito();
}
```

# Arquitectura MVC - Controlador

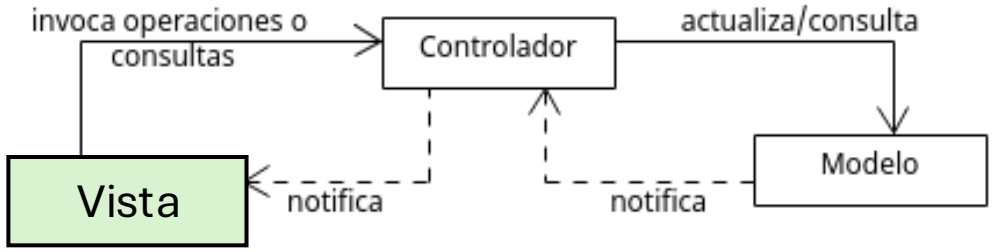
## **VISTA** - ¿Qué se hace ahí?




- Incluye:
  - La interfaz de usuario (p.ej. *.fxml)
    - Muestra la información que hay en el modelo
  - Clases que le dan soporte
    - Gestionan la interacción con el usuario (eventos)
    - Navegación entre pantallas
    - Controladores de FXML: para distinguirlos del controlador de negocio los llamaremos **ViewControllers**

# Arquitectura MVC - Controlador

## **VISTA** - ¿Qué se hace ahí?

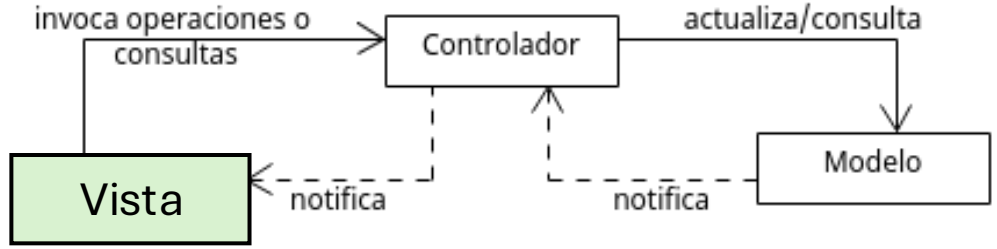



- Incluye:
  - La interfaz de usuario (p.ej. *.fxml)
    - Muestra la información que hay en el modelo
  - Clases que le dan soporte
    - Gestionan la interacción con el usuario (eventos)
    - Navegación entre pantallas
    - Controladores de FXML: para distinguirlos del controlador de negocio los llamaremos **ViewControllers**

**Recomendación:**
Encapsular este comportamiento en una clase (SceneManager)
(Mejora con respecto al anterior boletín donde teníamos todas las cargas dispersas!!!)

# Arquitectura MVC - Controlador

## **VISTA** - ¿Qué se hace ahí? – (Más JavaFX)




- Las vistas/visualizaciones pueden incluirse unas a otras
  - En JavaFX, uso de **fx:include**

Otra mejora con respecto al anterior boletín
Caso práctico: Reutilizar barra de menús

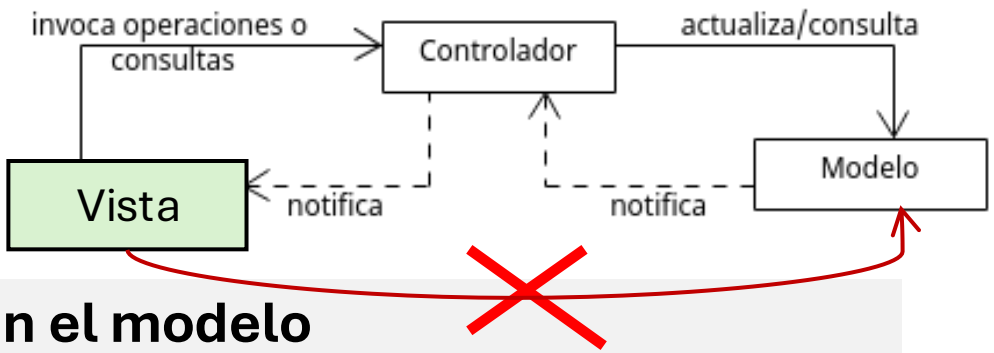
- Interacción con el usuario: Uso de diálogos (cuando **Alert** no es suficiente)

```
<DialogPane …>
        <content>…
        <buttonTypes>…
        <header>…
</DialogPane …>
```

```
DialogPane pane = (DialogPane) loadFXML(fxml);
Dialog<Void> dialog = new Dialog<>();
dialog.setDialogPane(pane);
dialog.setTitle(titulo);
dialog.initStyle(StageStyle.UTILITY);
// Muestra el diálogo y espera que el usuario presione
Aceptar
dialog.showAndWait();
```

# Arquitectura MVC - Controlador

## **Ejercicio 4** – Encuentra el fallo




**Importante: La vista nunca escribe directamente en el modelo**
(lo modifica siempre a través del controlador)

- **Objetivo**: Hay una parte de la vista donde se rompe esta regla ¡¡¡Encuéntrala!!!

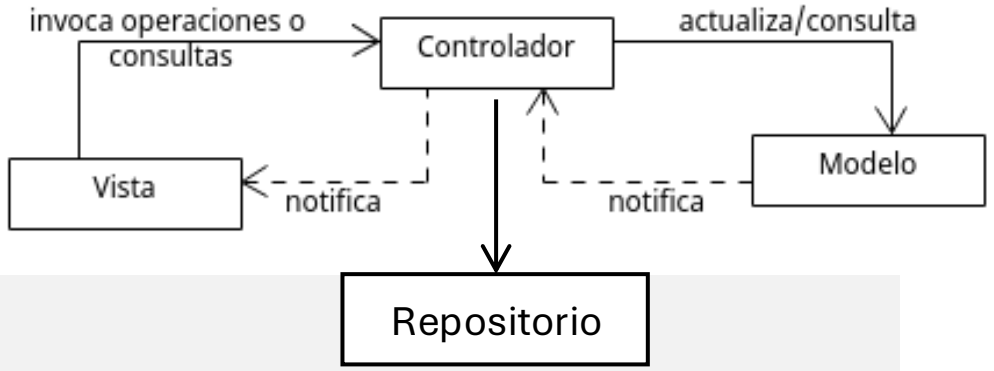


- ¿Podríamos modificar el modelo para que no sea posible esta modificación desde la vista (pero sí desde el modelo/controlador)?

Fuente:
https://despicableme.fandom.com/wiki/Carl?file=Carl.png

# Arquitectura MVC - Configuración

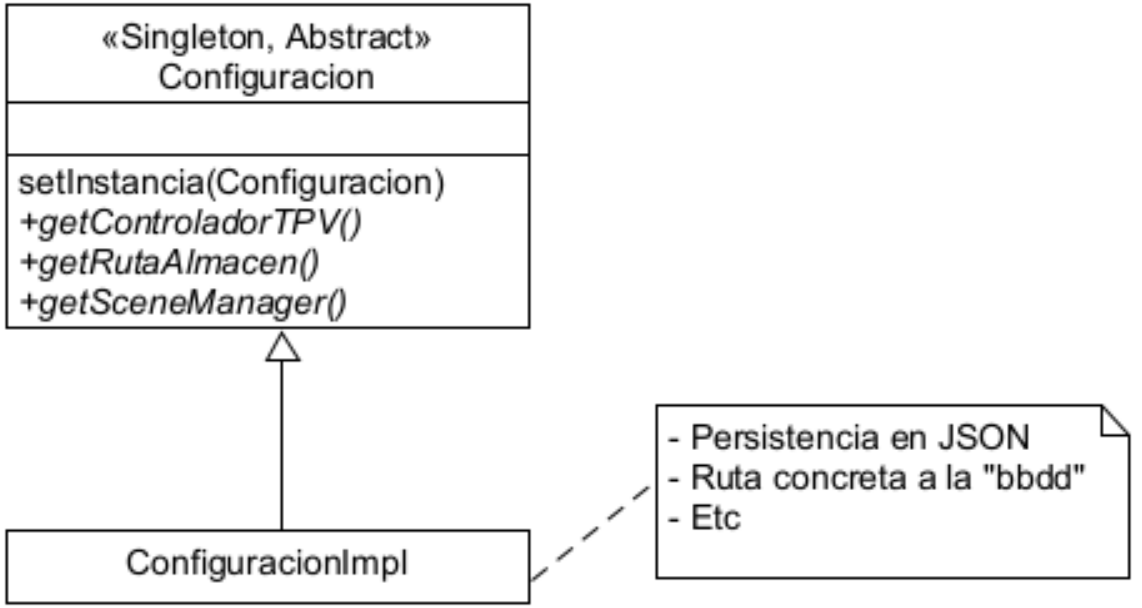
## Configuración de una instancia MVC concreta




- Una de las ventajas es que podemos reutilizar Componentes (cambiar de vista, de persistencia, Etc.)

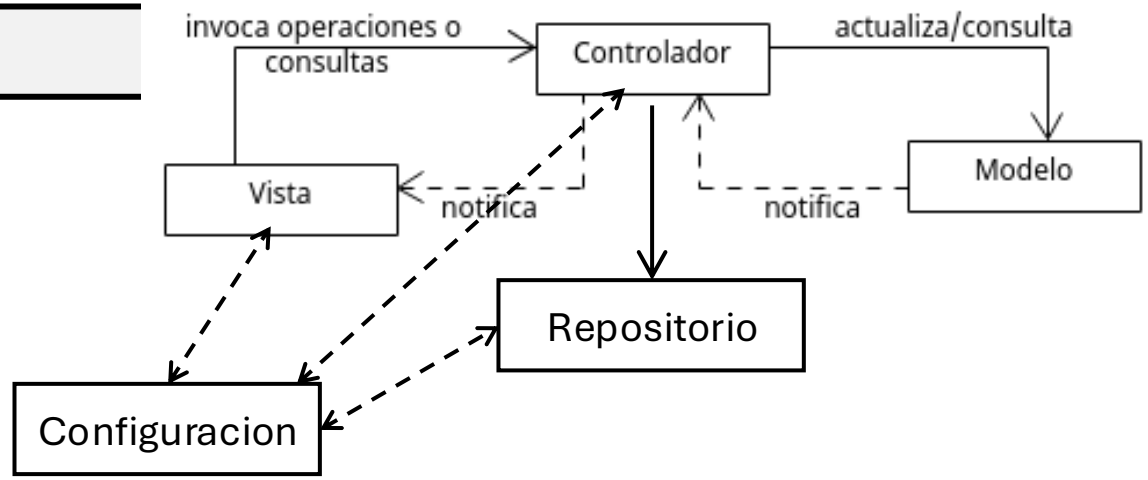
- ¿Cómo lanzamos una configuración concreta (P.ej. interfaz JavaFx + persistencia en memoria vs interfaz por comandos + persistencia Jackson)?

- **Solución: Contenedor de dependencias**



# Arquitectura MVC - Configuración

## **Ejercicio 5** – Ventajas de la configuración




- **Objetivo**: Discutir ejemplos en los que esta aproximación puede ser beneficiosa

# Arquitectura MVC y patrones

## MVC suele usar otros patrones de diseño

- Observer para notificaciones de cambios.
- Singleton /Abstract factory para configuración.
- Patrón repositorio para persistencia.
- Etc.

- **OJO:** Aplicar una buena arquitectura ayuda a utilizar mejor los patrones de diseño, pero no evita totalmente el usarlos mal!!!

# Arquitectura MVC y patrones

## **Ejercicio 6** – Violación de un patrón GRASP

- ¡¡¡El patrón experto se viola en una parte del código!!!
- **Objetivo:** Encontradlo y proponed los cambios necesarios para evitarlo